

Lógica de Programação

Prof.^a Kecia Aline Marques Ferreira

DECOM | Departamento de
Computação



Lógica de Programação

Comandos e recursos em C

Comandos de Desvio

- Atribuição composta
- Atribuição a mais de uma variável
- Definição de constantes
- Conversão de tipos e *cast*
- Comandos **break** e **continue**
- Operador condicional ternário **?:**
- Strings
- Bibliotecas de C

Atribuição Composta

Atribuição Composta

Operador	Operação	Exemplo	Equivalência
<code>+=</code>	Atribuição com soma	<code>a+=b</code>	<code>a = a + b</code>
<code>-=</code>	Atribuição com subtração	<code>a-=b</code>	<code>a = a - b</code>
<code>*=</code>	Atribuição com multiplicação	<code>a*=b</code>	<code>a = a * b</code>
<code>/=</code>	Atribuição com divisão	<code>a/=b</code>	<code>a = a / b</code>
<code>%=</code>	Atribuição com resto da divisão inteira	<code>a%=b</code>	<code>a = a % b</code>

Existem outras atribuições compostas envolvendo operações com bits. Aqui, foram mostradas as que envolvem operações aritméticas.

Atribuição a Mais de Uma Variável

Atribuição a Mais de uma Variável

Em C, é possível, com um único comando, realizar uma atribuição do valor resultante de uma expressão a mais de uma variável. A sintaxe é a seguinte:

var1 = var2 = var3 = expressão

A ordem de execução da atribuição é da direita para a esquerda, ou seja, a expressão é calculada, atribuída da var3, que é atribuída a var2, sucessivamente, até a primeira variável.

Ex.: `a = b = c = 10;`

Atribuição a Mais de uma Variável

```
#include <stdio.h>

void main(){
    int a, b, c, d;
    int x=10;

    a = b = c = d = x*5;

    printf("%d, %d, %d, %d, %d", a, b, c, d, x);
}
```

O programa imprime: 50, 50, 50, 50, 10

Definição de Constantes

Definição de Constantes

Uma forma de se definir valores constantes em um programa em C, é utilizar a diretiva do pré-processador **#define**

O que é o pré-processador?

É uma parte do Sistema de Processamento de Linguagens de C que prepara o arquivo para ser compilado.

O que é uma diretiva de pré-processador?

É uma instrução ao pré-processador sobre algo que ele deve realizar no programa a ser compilador

Definição de Constantes

A diretiva **#include**, por exemplo, informa a pré-processador o nome da biblioteca onde se encontram as definições das funções e recursos que o programa utiliza

```
#include <string.h>
```

A diretiva **#define** informa ao pré-processador que ele deve substituir a ocorrência da palavra definida pela constante informada

```
#define TAM 10
```

Definição de Constantes

```
#include <stdio.h>

#define TAXA 10

void main(){
    float saldo;

    printf("Informe o saldo: ");
    scanf("%f", &saldo);

    if (saldo<0)
        printf ("Saldo invalido;");
    else{
        saldo += saldo*TAXA/100;
        printf("O saldo final e %f.", saldo);
    }
}
```

Definição de Constantes

Outra forma de se definir valores constantes em um programa em C é utilizar a palavra **const**

Neste caso, será reservada uma posição de memória para a constante em tempo de execução, porém, ao contrários das variáveis, essa posição não poderá ter o seu valor alterado

Definição de Constantes

```
#include <stdio.h>

void main(){
    float saldo;
    const float TAXA=10;

    printf("Informe o saldo: ");
    scanf("%f", &saldo);

    if (saldo<0)
        printf ("Saldo invalido;");
    else{
        saldo += saldo*TAXA/100;
        printf("O saldo final e %f.", saldo);
    }
}
```

Conversão de Dados e *cast*

Conversão de Tipos e *cast*

Conversão de tipo é uma operação que o compilador realiza para converter o tipo de um dado

Por exemplo, em algumas situações, pode ser necessário tratar um dado inteiro como um real.

Há conversões de tipos que são realizadas implicitamente pelo compilador, ou seja, não é necessário realizar comando algum.

Outras demandam um comando explícito para realizar a conversão.

Conversão de Tipos e *cast*

Conversão implícita de tipos

São exemplos de conversões implícitas realizadas pelo compilador:

- Se um dos operandos de uma expressão aritmética for *float*, o outro operando é convertido para *float* e o resultado da operação é um *float*
- Um inteiro pode ser atribuído a um *float*. Nesse caso, o valor inteiro é convertido para *float*.
- Um *float* pode ser atribuído a um inteiro. Nesse caso, o valor *float* é convertido para inteiro, tomando-se a sua parte inteira como valor.
- Um caractere pode ser tratado com inteiro e vice-versa. Nesse caso, o valor inteiro é aquele correspondente ao caractere de acordo com a tabela ASCII

Conversão de Tipos e *cast*

Conversão explícita de tipos

Nos casos em que o compilador não realiza conversão implícita e em que se deseja realizar uma conversão, é necessária “forçá-la”, ou seja, defini-la explicitamente. Isso é denominado ***cast***

A sintaxe é:

(tipo) expressão

Exemplo: (float) a/b

Comandos break e continue

Comando **break**

Em um loop, o comando *break* força o seu encerramento, ou seja, o loop é imediatamente encerrado e é executada a próxima instrução após o loop.

Ex.: Um programa para pesquisar um elemento em um vetor e informar a sua posição. Se o elemento não for encontrado, o programa deverá exibir uma mensagem informativa.

Comando break

```
#include <stdio.h>
#define TAM 10

void main(){
    int vet[TAM], val, i;

    printf("\nEntre com os valores do vetor:");
    for (i=0; i<TAM; i++)
        scanf("%d", &vet[i]);

    printf("\nQual valor deseja pesquisar? ");
    scanf("%d", &val);

    for(i=0; i<TAM; i++)
        if (vet[i] == val) break;

    if(i<TAM) printf("\nO elemento esta na posicao %d", i);
    else printf("\nO elemento nao foi encontrado.");
}
```

Loop infinito

Um loop infinito ocorre quando a condição de parada do loop nunca é alcançada no programa

Isso pode ocorrer por um erro de lógica ou porque o programador propositalmente estabeleceu um loop infinito

No segundo caso, deve haver um comando *break* no corpo do loop

Embora a linguagem C permita que o programador faça isso, o recomendável é evitar esse tipo de situação

Loop infinito

Estrutura de loop infinito com comando while e do-while

```
while (1){  
    ...  
    if (condição) break;  
    ...  
}
```

```
do{  
    ...  
    if (condição) break;  
    ...  
}while (1);
```

Loop infinito

Estrutura de loop infinito com comando for

```
for(;;) {  
    ...  
    if (condição) break;  
    ...  
}
```


Comando **continue**

Em um loop, o comando *continue* força que o fluxo de execução do programa seja desviado para o início do loop imediatamente.

Ex.: O que o programa a seguir realiza?

Comando continue

```
#include <stdio.h>

void main() {

    char ch;

    while( (ch=getchar()) != '.' ) {

        if (ch >= '0' && ch <= '9') continue;
        printf("%c", ch);

    }
}
```

Comando **continue**

Lê uma sequência de caracteres finalizada por ponto (‘.’) e imprime os caracteres lidos, desconsiderando-se os dígitos.

Exemplo de execução do programa:

```
logica 123de progra789macao.  
logica de programacao
```

Operador Condicional Ternário

?:

O Operador Condicional Ternário ?:

Considere um comando condicional simples cuja execução resulta na definição do valor de uma variável.

```
if (a>b) maior=a;  
else maior=b;
```

O operador condicional ternário equivale a comandos como esses. Ele possui a seguinte sintaxe:

exp1 ? exp2 : exp3

Onde:

exp1 é uma expressão lógica

exp2 e exp3 são expressões

O comando correspondente ao comando if-else anterior é:

```
maior = (a>b)? a : b;
```

Strings

Strings

Uma string é uma cadeia de caracteres

Em C, uma string é definida com um vetor de caracteres

Ex.: `char nome[50];`

Define uma string de 50 caracteres

Uma string pode ser inicializada das seguintes formas:

```
char nome[50] = "Maria"
```

```
char nome[50] = {'M', 'a', 'r', 'i', 'a'};
```

Por definição em C, uma string é sempre terminada com o caractere `'\0'`.

Strings

Uma string pode ser lida das seguintes formas:

Utilizando a função *scanf*

Nesse caso:

- Utiliza-se %s (o mesmo para o *printf*)
- A função só lerá a sequência de caracteres até o primeiro espaço em branco, desconsiderando os demais caracteres digitados;
- Não se colocar o operador & antes do nome da string na função.

```
void main() {  
  
    char str[50];  
  
    printf("Digite uma string: ");  
    scanf("%s", str);  
  
    printf("A string digitada foi: %s", str);  
}
```


Strings

Utilizando a função *fgets*

Nesse caso:

- A string lida tem o tamanho máximo informado ao se chamar *fgets*

```
#include <stdio.h>

void main(){

    char str[50];

    printf("Digite uma string: ");
    fgets(str, 50, stdin);

    printf("A string digitada foi: %s", str);
}
```

Strings

Utilizando a função *gets*

Nesse caso:

- Deve-se incluir a biblioteca `conio.h`, mas essa biblioteca não faz parte da biblioteca padrão de C
- A string é lida por completo, até que seja digitado um `enter` pelo usuário.

```
void main() {  
  
    char str[50];  
  
    printf("Digite uma string: ");  
    gets(str);  
  
    printf("A string digitada foi: %s", str);  
}
```

Strings

Como uma string é um vetor, pode-se utilizar os mesmos procedimentos de varreduras de vetor para percorrer strings

Exemplo: Um programa para escrever uma string ao inverso

Strings

```
#include <stdio.h>
#include <string.h>

void main(){

    char str[50];

    printf("Digite uma string: ");
    gets(str);

    printf("\nA string digitada foi: %s\n", str);

    for (int i=strlen(str)-1; i>=0; i--) //strlen devolve o tamanho de str
        printf("%c", str[i]);
}
```

Strings

A biblioteca `string.h` possui funções úteis para manipulação de strings, dentre elas:

- `size_t strlen(const char* str)`

Retorna a quantidade de caracteres contidos na string

- `char* strcat(char* str1, char* str2)`

Concatena as strings `str1` e `str2` e armazena o resultado na string `str1`

- `char* strncat(char* str1, char* str2, size_t n)`

Concatena os primeiros *n* caracteres da string `str1` com a string `str2` e armazena o resultado na string `str1`

Strings

- `char* strcpy(char* str1, char* str2)`

Copia a string `str2` para a string `str1`. Caso a string `str2` seja maior do que a string `str1`, pode ocorrer erro em tempo de execução, causado pelo fato de acesso fora dos limites do tamanho da string

- `char* strncpy(char* str1, char* str2, size_t n)`

Copia os primeiros n caracteres da string `str2` para a string `str1` e armazena o resultado na string `str1`. Caso n seja maior do que o tamanho da string `str1`, pode ocorrer erro em tempo de execução

Strings

- `char* strcmp(char* str1, char* str2)`

Compara str1 com str2 lexicograficamente e devolve:

- valor negativo, caso str1 seja menor do que str2;
- zero, caso str1 seja igual a str2;
- valor positivo, caso str1 seja maior do que str2.

- `char* strncmp(char* str1, char* str2, size_t n)`

Compara str1 com str2 lexicograficamente, considerando os *n* primeiros caracteres das strings e devolve:

- valor negativo, caso str1 seja menor do que str2;
- zero, caso str1 seja igual a str2;
- valor positivo, caso str1 seja maior do que str2.

Se há menos do que *n* caracteres em alguma das strings, a comparação termina ao encontrar o primeiro `'\0'`

Biblioteca de C

Biblioteca de C

A biblioteca de C é extensa

Além das bibliotecas e funções já apresentadas na disciplina, há dezenas de outras

É importante consultar sempre o manual da linguagem C para se verificar as funções existentes e suas respectivas formas de uso

O manual da implementação da linguagem C dada pelo GCC (GNU C Compiler) está disponível em <https://www.gnu.org/software/libc/manual/pdf/libc.pdf>

Mas há outras diversas fontes disponíveis online e em livros.